

CSE220 Sampling

Online Exam Practice Workbook

Practical theory, 18 exam-style coding problems, and complete solutions

Primary source: Lecture 5 – Sampling
Focus: NumPy and Matplotlib implementation
Suggested time: 20–30 minutes per problem
Assumed imports: `numpy`, `matplotlib.pyplot`

How to use this workbook. Attempt one problem in a separate Python file before checking Part III. The problems deliberately resemble online-evaluation modifications: complete a small function, repair a pipeline, return specified arrays, or produce a diagnostic plot. Space is left after every prompt for iPad annotations.

Contents

1	Practical Theory and Coding Patterns	3
1.1	Uniform sampling	3
1.2	Nyquist condition and aliasing	3
1.3	Why aliasing appears in the spectrum	3
1.4	Reconstruction methods	3
1.5	Downsampling and anti-aliasing	4
1.6	DFT bins and physical frequency	4
1.7	Fast exam checklist	4
2	Practice Problems	5
	Problem 1: Sampling Grid and Nyquist Report	5
	Problem 2: Alias Table	5
	Problem 3: Same Samples, Different Continuous Tones	5
	Problem 4: FFT Peak to Physical Frequency	5
	Problem 5: Signed DFT Bin Map	6
	Problem 6: Spectral Copy Intervals	6
	Problem 7: Downsampling and Predicted Aliases	6
	Problem 8: Anti-Alias Filter Before Decimation	6
	Problem 9: Zero-Order Hold Reconstruction	6
	Problem 10: Endpoint-Safe Linear Reconstruction	6
	Problem 11: Manual Linear Interpolation	7
	Problem 12: Vectorized Sinc Reconstruction	7
	Problem 13: Compare ZOH, Linear, and Sinc	7
	Problem 14: Zero-Insertion Upsampling	7
	Problem 15: Interpolation Filter After Upsampling	7
	Problem 16: Rational Resampling by Linear Interpolation	8
	Problem 17: Periodic Signal and DFT Coefficients	8
	Problem 18: Repair a Sampling Pipeline	8
3	Worked Solutions	9
	Solution 1: Sampling Grid and Nyquist Report	9
	Solution 2: Alias Table	9
	Solution 3: Same Samples, Different Continuous Tones	9
	Solution 4: FFT Peak to Physical Frequency	10
	Solution 5: Signed DFT Bin Map	10
	Solution 6: Spectral Copy Intervals	10
	Solution 7: Downsampling and Predicted Aliases	11
	Solution 8: Anti-Alias Filter Before Decimation	11

Solution 9: Zero-Order Hold Reconstruction	11
Solution 10: Endpoint-Safe Linear Reconstruction	12
Solution 11: Manual Linear Interpolation	12
Solution 12: Vectorized Sinc Reconstruction	12
Solution 13: Compare ZOH, Linear, and Sinc	13
Solution 14: Zero-Insertion Upsampling	13
Solution 15: Interpolation Filter After Upsampling	14
Solution 16: Rational Resampling by Linear Interpolation	14
Solution 17: Periodic Signal and DFT Coefficients	14
Solution 18: Repair a Sampling Pipeline	15
4 Final Revision Sheet	15

1 Practical Theory and Coding Patterns

1.1 Uniform sampling

A continuous-time signal is represented by values read every T seconds:

$$x[n] = x(nT), \quad f_s = \frac{1}{T}.$$

For a duration D , a reliable grid is `t = np.arange(N) / fs` with `N = int(round(D*fs))`. This makes the intended number of samples explicit and avoids endpoint surprises.

```
1 N = int(round(duration * fs))
2 t = np.arange(N) / fs
3 x = np.sin(2*np.pi*f0*t)
```

1.2 Nyquist condition and aliasing

If $x(t)$ is band-limited to $f_{\max}$ Hz, exact recovery requires

$$f_s > 2f_{\max}.$$

The lecture uses the equivalent angular-frequency form $\omega_s > 2\omega_M$. Equality is a fragile boundary case; use a strict inequality in code. The Nyquist rate is $2f_{\max}$, while the Nyquist frequency of a sampler is $f_s/2$.

After sampling, frequencies that differ by integer multiples of f_s are indistinguishable. A tone can be folded to the nonnegative baseband by

```
1 def alias_frequency(f, fs):
2     return abs(((f + fs/2) % fs) - fs/2)
```

For real sinusoids, also remember that folding can change phase or sign even when the magnitude spectrum shows the same nonnegative frequency.

1.3 Why aliasing appears in the spectrum

Impulse-train sampling creates shifted copies of the original spectrum:

$$X_p(j\omega) = \frac{1}{T} \sum_{k=-\infty}^{\infty} X(j(\omega - k\omega_s)), \quad \omega_s = \frac{2\pi}{T}.$$

Each copy is centered at $k\omega_s$ and, for a band limit ω_M , occupies $[k\omega_s - \omega_M, k\omega_s + \omega_M]$. Copies overlap when $\omega_s < 2\omega_M$. Code questions usually ask you to calculate these intervals or visualize a few copies, not derive the formula.

1.4 Reconstruction methods

Ideal reconstruction is a sum of shifted sinc functions:

$$x_r(t) = \sum_n x[n] \operatorname{sinc}\left(\frac{t - nT}{T}\right),$$

where NumPy's `np.sinc(u)` means $\sin(\pi u)/(\pi u)$. It is exact for an infinite, Nyquist-sampled band-limited signal, but a finite record produces edge/truncation error.

Method	Meaning	Core NumPy pattern
Zero insertion	Put $L - 1$ zeros between original samples	<code>y[:L] = x</code>
Zero-order hold	Hold each value until the next sample	<code>np.repeat(x, L)</code>
Linear interpolation	Join adjacent samples by straight lines	<code>np.interp(new_t, old_t, x)</code>
Sinc interpolation	Sum one shifted sinc per sample	vectorized sum or loop over n

Zero insertion alone is not reconstruction. It creates empty high-rate sample positions; an interpolation low-pass filter must follow. ZOH uses a rectangular interpolation pulse and has a sinc-shaped frequency response. Linear interpolation uses a triangular pulse and a sinc-squared magnitude response, so it suppresses high-frequency copies faster than ZOH.

1.5 Downsampling and anti-aliasing

Keeping every M th sample changes the rate to f_s/M :

```
1 y = x[:,M]
2 new_fs = fs / M
```

Before decimation, frequencies above the new Nyquist frequency must be removed. In practice, low-pass filter first, then take every M th sample. A short windowed-sinc FIR is sufficient for an exam-scale implementation.

1.6 DFT bins and physical frequency

For N samples taken at f_s Hz,

$$\Delta f = \frac{f_s}{N}, \quad f_k = \frac{k f_s}{N}.$$

For even N , bins 0 through $N/2$ cover dc through the Nyquist frequency. Higher indices wrap to negative frequencies. Prefer `np.fft.fftfreq(N, d=1/fs)` when signed frequencies are needed, and `np.fft.rfftfreq` with `np.fft.rfft` for real signals.

If a periodic continuous signal is sampled exactly N times per period and satisfies Nyquist, its DFT bins are scaled Fourier-series coefficients: $X[k] = N a_k$ under the lecture's convention. Coherent sampling (an integer number of cycles in the record) places a tone exactly on DFT bins and avoids leakage.

1.7 Fast exam checklist

- Identify the old and new sampling rates before writing array slices.
- Use $f_s > 2f_{\max}$, not $f_s \geq 2f_{\max}$, for the lecture's exact-recovery condition.
- Keep time and signal arrays the same length; recompute time after rate changes.
- Use `abs(FFT)` to locate magnitude peaks and a frequency array to label them.
- Filter before downsampling; filter after zero-insertion upsampling.
- Treat finite-record sinc reconstruction errors near the ends as truncation effects.

2 Practice Problems

Unless a prompt says otherwise, use only `numpy` and `matplotlib.pyplot`. Functions should return values as specified so they can be unit-tested.

Problem 1 Sampling Grid and Nyquist Report

Complete the function below. It must create exactly `round(fs*duration)` samples of $\sum_i A_i \sin(2\pi f_i t)$ and return `t`, `x`, `report`. The report is a dictionary containing `fmax`, `nyquist_rate`, and `safe`; `safe` follows the lecture's strict condition.

```
1 def sample_signal(freqs, amps, fs, duration):
2     # TODO 1: validate matching, nonempty lists and positive fs/duration
3     # TODO 2: create the sampling grid and composite signal
4     # TODO 3: build the report
5     pass
```

Test with frequencies 30, 80, and 140 Hz at $f_s = 400$ Hz.

Problem 2 Alias Table

Write `alias_table(freqs, fs)` returning an array with one row per input tone: `[original, folded, aliases]`. Here `folded` is in $[0, f_s/2]$ and `aliases` is 1 when the original frequency is outside that band. Test `[40, 260, 540, 760]` at $f_s = 500$ Hz.

Problem 3 Same Samples, Different Continuous Tones

At $f_s = 1000$ Hz, generate one second of samples for 100 Hz and 900 Hz sines. Return the maximum of `abs(x1 + x2)` and explain why addition, rather than subtraction, is used for these sine phases. Then repeat with cosines and return the maximum of `abs(c1 - c2)`.

Problem 4 FFT Peak to Physical Frequency

Complete a function that takes a real array `x` and `fs`, removes its mean, computes the one-sided DFT, and returns the three strongest non-dc peak frequencies in descending magnitude order. Use `rfft` and `rfftfreq`; do not manually assume a 1 Hz bin width.

Problem 5 Signed DFT Bin Map

Write `bin_map(N, fs)` returning an $N \times 2$ array whose rows are `[k, signed_frequency]`. Verify that for $N = 8$, $f_s = 800$ Hz, bin 5 means -300 Hz. Also write `frequency_to_bin(f, N, fs)` for frequencies that lie exactly on a bin, returning the wrapped index in $0, \dots, N - 1$.

Problem 6 Spectral Copy Intervals

Given `fmax` and `fs`, return the intervals occupied by spectrum copies centered at kf_s for $k = -2, -1, 0, 1, 2$. Also return whether the baseband copy overlaps either neighbor. Test $(f_{\max}, f_s) = (120, 300)$ and $(120, 200)$ Hz.

Problem 7 Downsampling and Predicted Aliases

A signal sampled at 1200 Hz contains 80, 260, and 430 Hz. Downsample by $M = 3$ without filtering. Return the new rate, the downsampled array, and the predicted folded frequency of each component. Confirm the prediction with an `rfft` magnitude plot.

Problem 8 Anti-Alias Filter Before Decimation

Implement a Hamming-windowed sinc low-pass filter of odd length `numtaps`. Its cutoff is given in Hz. Normalize the coefficients to unit dc gain. Apply it before the factor-3 downsampling in Problem 7, using a cutoff of 180 Hz and 101 taps. Plot before/after spectra and explain which original components remain.

Problem 9 Zero-Order Hold Reconstruction

Complete `zoh_reconstruct(samples, fs, L)`. Return a high-rate time array and values obtained by holding each sample for L output points. The output rate is Lf_s . State the output length and final time for five input samples, $f_s = 10$ Hz, $L = 4$.

Problem 10 Endpoint-Safe Linear Reconstruction

Write `linear_reconstruct(samples, fs, new_fs)` using `np.interp`. The new grid must begin at zero and include the time of the final original sample exactly. Do not extrapolate beyond it. Test with `samples=[0,2,1]`, $f_s = 2$ Hz, $f'_s = 8$ Hz.

Problem 11 Manual Linear Interpolation

Reimplement Problem 10 without `np.interp`. For each requested time, find the left sample index, compute α , and use $(1 - \alpha)x[n] + \alpha x[n + 1]$. Handle the last endpoint safely. Compare your result with NumPy using `np.allclose`.

Problem 12 Vectorized Sinc Reconstruction

Implement `sinc_reconstruct(samples, fs, t_new)` in two ways: (a) a loop over input samples that updates the entire result array and (b) a fully vectorized sinc matrix. Return both outputs and verify they agree. Explain the shapes used in the matrix version.

Problem 13 Compare ZOH, Linear, and Sinc

Sample a 5 Hz sine for 1 second at 40 Hz, then reconstruct on a 1000 Hz grid using ZOH, linear interpolation, and sinc interpolation. Compute RMSE against the known sine only on $0 \leq t \leq (N - 1)/f_s$. Return the three errors and plot a zoomed 0.2-second segment. Which method should normally win here, and why might finite-record sinc be imperfect near the ends?

Problem 14 Zero-Insertion Upsampling

Write `upsample_zeros(x, L)` that places the original values every L samples and zeros elsewhere. Return the new array and a Boolean mask marking original-sample positions. For `x=[2,5,3]`, $L = 3$, verify both outputs. Explain why this is neither ZOH nor linear interpolation.

Problem 15 Interpolation Filter After Upsampling

Upsample a 30 Hz sine from 200 Hz to 800 Hz by zero insertion ($L = 4$). Design a 65-tap Hamming-windowed sinc filter with cutoff 90 Hz. Because zero insertion reduces the discrete sequence's dc gain, scale the filter by L . Filter with `mode='same'` and compare the middle portion with the true 30 Hz samples at 800 Hz using RMSE.

Problem 16 Rational Resampling by Linear Interpolation

Write `resample_linear(x, fs_old, p, q)` for the rate $f_{s,new} = f_{s,old}p/q$. Use the same physical end time as the input and include the last original time. Return `new_t, y, new_fs`. Test 100 samples at 1000 Hz with $p = 3, q = 2$. This is an exam-scale time-domain resampler; mention why production resampling also needs filtering.

Problem 17 Periodic Signal and DFT Coefficients

One period is sampled with $N = 32$ points:

$$x(t) = 2 + 3 \cos(2\pi t/T_0) - 1.5 \sin(4\pi t/T_0).$$

Compute its DFT and report all bins with magnitude above 10^{-9} . Verify the lecture connection $X[k] = Na_k$ by stating the expected complex values at bins $0, 1, 2, N - 2, N - 1$.

Problem 18 Repair a Sampling Pipeline

The function below has multiple conceptual or array-grid errors. Repair it so it samples a composite signal, safely decimates by M , and reports the dominant non-dc frequency after decimation.

```

1 def analyze_after_downsampling(freqs, fs, duration, M):
2     t = np.arange(0, duration, fs)
3     x = sum(np.sin(2*np.pi*f*t) for f in freqs)
4     y = x[::M]
5     new_fs = fs * M
6     Y = np.fft.rfft(y)
7     f = np.arange(len(Y)) * fs / len(Y)
8     peak = f[np.argmax(np.abs(Y))]
9     return y, new_fs, peak

```

Use a windowed-sinc anti-alias filter, create the correct frequency axis, ignore dc, and validate inputs. Choose a cutoff slightly below the new Nyquist frequency.

3 Worked Solutions

Solution 1 Sampling Grid and Nyquist Report

```

1 import numpy as np
2
3 def sample_signal(freqs, amps, fs, duration):
4     freqs = np.asarray(freqs, dtype=float)
5     amps = np.asarray(amps, dtype=float)
6     if freqs.size == 0 or freqs.shape != amps.shape:
7         raise ValueError("freqs and amps must be nonempty and match")
8     if fs <= 0 or duration <= 0 or np.any(freqs < 0):
9         raise ValueError("rates/duration must be positive; freqs nonnegative")
10    N = int(round(fs * duration))
11    t = np.arange(N) / fs
12    x = np.sum(amps[:, None] * np.sin(2*np.pi*freqs[:, None]*t), axis=0)
13    fmax = float(np.max(freqs))
14    report = {"fmax": fmax, "nyquist_rate": 2*fmax,
15             "safe": bool(fs > 2*fmax)}
16    return t, x, report

```

For the test, $f_{\max} = 140$ Hz, the Nyquist rate is 280 Hz, and 400 Hz is safe.

Solution 2 Alias Table

```

1 def alias_table(freqs, fs):
2     freqs = np.asarray(freqs, dtype=float)
3     if fs <= 0 or np.any(freqs < 0):
4         raise ValueError("fs must be positive and frequencies nonnegative")
5     folded = np.abs(((freqs + fs/2) % fs) - fs/2)
6     aliased = (freqs > fs/2).astype(int)
7     return np.column_stack((freqs, folded, aliased))
8
9 print(alias_table([40, 260, 540, 760], 500))
10 # [40,40,0], [260,240,1], [540,40,1], [760,240,1]

```

The modulo operation first wraps frequencies into one f_s -wide period; the absolute value folds the signed baseband to nonnegative frequency.

Solution 3 Same Samples, Different Continuous Tones

```

1 fs = 1000
2 t = np.arange(fs) / fs
3 x1 = np.sin(2*np.pi*100*t)
4 x2 = np.sin(2*np.pi*900*t)
5 sine_error = np.max(np.abs(x1 + x2))
6
7 c1 = np.cos(2*np.pi*100*t)
8 c2 = np.cos(2*np.pi*900*t)
9 cos_error = np.max(np.abs(c1 - c2))

```

At integer n , $\sin(2\pi(900/1000)n) = \sin(2\pi n - 2\pi(100/1000)n) = -\sin(2\pi(100/1000)n)$, so the sine sequences are negatives. Cosine is even, so the cosine sequences match. Both errors are near floating-point zero.

Solution 4 FFT Peak to Physical Frequency

```

1 def strongest_frequencies(x, fs, count=3):
2     x = np.asarray(x, dtype=float)
3     if x.size < 2 or fs <= 0 or count < 1:
4         raise ValueError("invalid input")
5     X = np.fft.rfft(x - np.mean(x))
6     f = np.fft.rfftfreq(x.size, d=1/fs)
7     mag = np.abs(X)
8     mag[0] = -np.inf
9     count = min(count, mag.size - 1)
10    idx = np.argpartition(mag, -count)[-count:]
11    idx = idx[np.argsort(mag[idx])[:-1]]
12    return f[idx]

```

This selects bins, not necessarily distinct spectral lobes. Online questions with coherent tones generally make that distinction harmless.

Solution 5 Signed DFT Bin Map

```

1 def bin_map(N, fs):
2     if N < 1 or fs <= 0:
3         raise ValueError("N and fs must be positive")
4     k = np.arange(N)
5     f = np.fft.fftfreq(N, d=1/fs)
6     return np.column_stack((k, f))
7
8 def frequency_to_bin(f, N, fs):
9     k_float = f * N / fs
10    k = int(round(k_float))
11    if not np.isclose(k_float, k):
12        raise ValueError("frequency is not exactly on a DFT bin")
13    return k % N

```

Here $\Delta f = 100$ Hz. Bin 5 wraps to $5 - 8 = -3$, hence -300 Hz.

Solution 6 Spectral Copy Intervals

```

1 def spectral_copy_intervals(fmax, fs):
2     if fmax < 0 or fs <= 0:
3         raise ValueError("invalid frequencies")
4     k = np.arange(-2, 3)
5     centers = k * fs
6     intervals = np.column_stack((k, centers-fmax, centers+fmax))
7     overlaps = fs < 2*fmax
8     return intervals, overlaps

```

At 300 Hz the copies have a 60 Hz guard gap because $300 > 240$. At 200 Hz they overlap by 40 Hz because $200 < 240$.

Solution 7 Downsampling and Predicted Aliases

```

1 import matplotlib.pyplot as plt
2
3 def downsample_and_predict(x, fs, component_freqs, M):
4     if M < 1 or int(M) != M:
5         raise ValueError("M must be a positive integer")
6     y = np.asarray(x)[::int(M)]
7     new_fs = fs / M
8     f = np.asarray(component_freqs, dtype=float)
9     aliases = np.abs(((f + new_fs/2) % new_fs) - new_fs/2)
10    return new_fs, y, aliases
11
12 fs = 1200; duration = 1.0; M = 3
13 t = np.arange(int(fs*duration)) / fs
14 tones = np.array([80, 260, 430])
15 x = np.sum(np.sin(2*np.pi*tones[:, None]*t), axis=0)
16 new_fs, y, aliases = downsample_and_predict(x, fs, tones, M)
17 fy = np.fft.rfftfreq(len(y), d=1/new_fs)
18 plt.plot(fy, np.abs(np.fft.rfft(y)))
19 plt.xlabel("Frequency (Hz)"); plt.ylabel("Magnitude"); plt.show()

```

The new rate is 400 Hz. The predicted frequencies are 80, 140, and 30 Hz respectively.

Solution 8 Anti-Alias Filter Before Decimation

```

1 def lowpass_fir(cutoff, fs, numtaps):
2     if not (0 < cutoff < fs/2) or numtaps < 3 or numtaps % 2 == 0:
3         raise ValueError("cutoff must be inside Nyquist; taps must be odd")
4     n = np.arange(numtaps) - (numtaps-1)/2
5     h = 2*cutoff/fs * np.sinc(2*cutoff*n/fs)
6     h *= np.hamming(numtaps)
7     return h / np.sum(h)
8
9 h = lowpass_fir(180, 1200, 101)
10 x_filtered = np.convolve(x, h, mode="same")
11 y_safe = x_filtered[::3]
12
13 def plot_spectrum(sig, rate, label):
14     f = np.fft.rfftfreq(len(sig), d=1/rate)
15     plt.plot(f, np.abs(np.fft.rfft(sig)), label=label)
16
17 plot_spectrum(x, 1200, "before")
18 plot_spectrum(y_safe, 400, "filtered then decimated")
19 plt.xlim(0, 500); plt.legend(); plt.show()

```

Only the 80 Hz component lies well inside the chosen passband. The 260 and 430 Hz components are strongly suppressed before they can fold into 140 and 30 Hz. A real FIR has a transition band, so attenuation is not an ideal brick wall.

Solution 9 Zero-Order Hold Reconstruction

```

1 def zoh_reconstruct(samples, fs, L):
2     samples = np.asarray(samples, dtype=float)
3     if fs <= 0 or L < 1 or int(L) != L:
4         raise ValueError("invalid fs or L")
5     L = int(L)

```

```

6     y = np.repeat(samples, L)
7     new_fs = L * fs
8     t_new = np.arange(len(y)) / new_fs
9     return t_new, y

```

Five samples produce 20 output points. At 40 Hz, the final time is $19/40 = 0.475$ s. The repeated final value extends beyond the original sample instant at 0.4 s, which is exactly the hold behavior.

Solution 10 Endpoint-Safe Linear Reconstruction

```

1 def linear_reconstruct(samples, fs, new_fs):
2     samples = np.asarray(samples, dtype=float)
3     if samples.size == 0 or fs <= 0 or new_fs <= 0:
4         raise ValueError("invalid input")
5     old_t = np.arange(samples.size) / fs
6     duration = old_t[-1]
7     count = int(round(duration * new_fs)) + 1
8     new_t = np.linspace(0.0, duration, count)
9     y = np.interp(new_t, old_t, samples)
10    return new_t, y

```

The test gives times from 0 through 1 s in steps of 0.125 s and necessarily includes the last original point (1,1).

Solution 11 Manual Linear Interpolation

```

1 def linear_reconstruct_manual(samples, fs, new_fs):
2     x = np.asarray(samples, dtype=float)
3     old_t = np.arange(len(x)) / fs
4     duration = old_t[-1]
5     new_t = np.linspace(0, duration, int(round(duration*new_fs))+1)
6     y = np.empty_like(new_t)
7     for i, t in enumerate(new_t):
8         if np.isclose(t, duration):
9             y[i] = x[-1]
10            continue
11            n = min(int(np.floor(t*fs)), len(x)-2)
12            alpha = (t - old_t[n]) / (old_t[n+1] - old_t[n])
13            y[i] = (1-alpha)*x[n] + alpha*x[n+1]
14    return new_t, y
15
16 t1, y1 = linear_reconstruct([0,2,1], 2, 8)
17 t2, y2 = linear_reconstruct_manual([0,2,1], 2, 8)
18 assert np.allclose(t1, t2) and np.allclose(y1, y2)

```

Solution 12 Vectorized Sinc Reconstruction

```

1 def sinc_reconstruct(samples, fs, t_new):
2     x = np.asarray(samples, dtype=float)
3     t_new = np.asarray(t_new, dtype=float)
4     T = 1/fs
5
6     loop_result = np.zeros_like(t_new)
7     for k in range(len(x)):
8         loop_result += x[k] * np.sinc((t_new-k*T)/T)

```

```

9
10 sample_t = np.arange(len(x)) / fs
11 S = np.sinc((t_new[:, None] - sample_t[None, :]) / T)
12 matrix_result = S @ x
13 return loop_result, matrix_result

```

If `t_new` has P entries and `samples` has N , the difference grid and sinc matrix are $P \times N$. Multiplying by the N -vector sums all sample contributions for every requested time. The loop version also updates all P outputs at once; NumPy performs the hidden inner loop.

Solution 13 Compare ZOH, Linear, and Sinc

```

1 fs = 40; dense_fs = 1000; f0 = 5; duration = 1.0
2 t = np.arange(int(fs*duration)) / fs
3 x = np.sin(2*np.pi*f0*t)
4 t_end = t[-1]
5 td = np.linspace(0, t_end, int(round(t_end*dense_fs))+1)
6 truth = np.sin(2*np.pi*f0*td)
7
8 # ZOH evaluated directly on td
9 indices = np.minimum((td*fs).astype(int), len(x)-1)
10 zoh = x[indices]
11 linear = np.interp(td, t, x)
12 _, sinc_y = sinc_reconstruct(x, fs, td)
13
14 rmse = lambda y: np.sqrt(np.mean((y-truth)**2))
15 errors = {"zoh": rmse(zoh), "linear": rmse(linear), "sinc": rmse(sinc_y)}
16
17 mask = td <= 0.2
18 plt.plot(td[mask], truth[mask], label="true")
19 plt.step(td[mask], zoh[mask], where="post", label="ZOH")
20 plt.plot(td[mask], linear[mask], label="linear")
21 plt.plot(td[mask], sinc_y[mask], label="sinc")
22 plt.legend(); plt.show()

```

Sinc should normally give the smallest interior error because 5 Hz is below the 20 Hz Nyquist frequency. The finite sum assumes unavailable samples outside the record are zero, so truncation harms the edges.

Solution 14 Zero-Insertion Upsampling

```

1 def upsample_zeros(x, L):
2     x = np.asarray(x)
3     if L < 1 or int(L) != L:
4         raise ValueError("L must be a positive integer")
5     L = int(L)
6     y = np.zeros(len(x)*L, dtype=np.result_type(x, float))
7     mask = np.zeros(len(y), dtype=bool)
8     y[:L] = x
9     mask[:L] = True
10    return y, mask
11
12 y, mask = upsample_zeros([2,5,3], 3)
13 assert np.array_equal(y, [2,0,0,5,0,0,3,0,0])
14 assert np.array_equal(mask, [1,0,0,1,0,0,1,0,0])

```

Zeros are placeholders. ZOH would repeat values; linear interpolation would calculate nonzero points on connecting lines.

Solution 15 Interpolation Filter After Upsampling

```

1 fs = 200; L = 4; new_fs = fs*L; f0 = 30; duration = 1.0
2 t = np.arange(int(fs*duration)) / fs
3 x = np.sin(2*np.pi*f0*t)
4 u, _ = upsample_zeros(x, L)
5
6 h = L * lowpass_fir(90, new_fs, 65)
7 y = np.convolve(u, h, mode="same")
8 tu = np.arange(len(u)) / new_fs
9 truth = np.sin(2*np.pi*f0*tu)
10
11 delay_margin = len(h) // 2
12 middle = slice(delay_margin, len(y)-delay_margin)
13 rmse = np.sqrt(np.mean((y[middle]-truth[middle])**2))
14 print(rmse)

```

The low-pass filter removes the spectral images created by zero insertion. Scaling by L restores the amplitude at original sample positions in the ideal interpolation convention. Edge samples are excluded because convolution padding creates transients.

Solution 16 Rational Resampling by Linear Interpolation

```

1 def resample_linear(x, fs_old, p, q):
2     x = np.asarray(x, dtype=float)
3     if x.size == 0 or fs_old <= 0 or p < 1 or q < 1:
4         raise ValueError("invalid input")
5     new_fs = fs_old * p / q
6     old_t = np.arange(len(x)) / fs_old
7     end = old_t[-1]
8     new_t = np.linspace(0, end, int(round(end*new_fs))+1)
9     y = np.interp(new_t, old_t, x)
10    return new_t, y, new_fs

```

The new rate is 1500 Hz. The original end time is 0.099 s, so the endpoint-inclusive grid contains approximately $0.099(1500) + 1$, rounded as coded. Linear interpolation is not an anti-alias filter; production rational resampling uses low-pass filtering to suppress images and prevent folding.

Solution 17 Periodic Signal and DFT Coefficients

```

1 N = 32
2 n = np.arange(N)
3 x = 2 + 3*np.cos(2*np.pi*n/N) - 1.5*np.sin(4*np.pi*n/N)
4 X = np.fft.fft(x)
5 active = np.flatnonzero(np.abs(X) > 1e-9)
6 reported = [(int(k), X[k]) for k in active]
7 print(reported)

```

The dc coefficient is $a_0 = 2$. The cosine gives $a_1 = a_{-1} = 3/2$. Since $-1.5 \sin(2\theta) = 0.75je^{j2\theta} - 0.75je^{-j2\theta}$, $a_2 = 0.75j$ and $a_{-2} = -0.75j$. Therefore

$$X[0] = 64, \quad X[1] = 48, \quad X[2] = 24j, \quad X[30] = -24j, \quad X[31] = 48.$$

Only these five bins exceed the threshold, apart from numerical noise.

Solution 18 Repair a Sampling Pipeline

```

1 def analyze_after_downsampling(freqs, fs, duration, M, numtaps=101):
2     freqs = np.asarray(freqs, dtype=float)
3     if freqs.size == 0 or np.any(freqs < 0):
4         raise ValueError("frequencies must be nonempty and nonnegative")
5     if fs <= 0 or duration <= 0 or M < 1 or int(M) != M:
6         raise ValueError("invalid sampling parameters")
7     M = int(M)
8     new_fs = fs / M
9     if numtaps % 2 == 0:
10        raise ValueError("numtaps must be odd")
11
12    N = int(round(fs*duration))
13    t = np.arange(N) / fs
14    x = np.sum(np.sin(2*np.pi*freqs[:, None]*t), axis=0)
15
16    cutoff = 0.9 * (new_fs/2)
17    h = lowpass_fir(cutoff, fs, numtaps)
18    filtered = np.convolve(x, h, mode="same")
19    y = filtered[:, :M]
20
21    Y = np.fft.rfft(y - np.mean(y))
22    f = np.fft.rfftfreq(len(y), d=1/new_fs)
23    if len(Y) < 2:
24        raise ValueError("record is too short for a non-dc peak")
25    peak_index = 1 + np.argmax(np.abs(Y[1:]))
26    return y, new_fs, float(f[peak_index])

```

The original used f_s as a time step, multiplied the sampling rate during decimation, and labeled bins with the wrong rate and denominator. The repair filters before slicing, uses f_s/M , constructs the axis with `rfftfreq`, and excludes dc from the peak search. The 0.9 factor leaves a modest transition guard below the new Nyquist boundary.

4 Final Revision Sheet

Prompt clue	First coding move
“sample for D seconds”	<code>N=round(fs*D); t=np.arange(N)/fs</code>
“will it alias?”	compare <code>fs > 2*fmax</code>
“what frequency is observed?”	fold modulo f_s into $[0, f_s/2]$
“DFT bin in Hz”	<code>np.fft.fftfreq</code> or <code>rfftfreq</code>
“downsample by M ”	low-pass, then <code>x[:, :M]</code> ; new rate f_s/M
“upsample by L ”	zeros at high-rate positions, then interpolation LPF
“staircase DAC”	<code>np.repeat</code> ; this is ZOH
“connect samples”	<code>np.interp</code> ; this is linear interpolation
“ideal reconstruction”	shifted <code>np.sinc</code> sum
“periodic one-cycle record”	expect harmonic DFT lines and $X[k] = Na_k$

Scope note. The theory and terminology follow Lecture 5 – Sampling. Decimation, zero-insertion upsampling, windowed-sinc FIR filtering, and rational resampling are included only as direct, exam-level coding extensions of alias prevention and reconstruction.